Faculty of Science and Engineering
School of Computing and Information Science

MOD006568
Neural Computing & Deel Learning

Element 010
Element Report

SID NUMBER: 2128741

Github Link: https://github.com/jugal-chawda/MOD006568-Neural-Computing-and-Deep-Learning

Submission deadline: 01/05/2026

Neural Network Models for Image Classification and Time Series Prediction

Contents

Part A – Cassava Leaf Disease Detection 2

Model 1: Custom Convolutional Neural Network (CNN) 4

Part B – Time Series Forecasting (Predicting Stock Prices) 6

3.4.4 Early Stopping 8

Early stopping was used during training to prevent overfitting. Training was stopped when validation loss stopped improving for several epochs. 8

RMSE provides a clear measure of prediction error and indicates how closely the model predictions match actual values. 9

4. Comparison and Discussion 9

5. Conclusion 10

References 10

Part A – Cassava Leaf Disease Detection

2.1 Problem Description

Cassava is one of the most important food crops in sub-Saharan Africa, and it feeds hundreds of millions of people. However, the crop is highly vulnerable to a variety of viral and bacterial diseases that can significantly reduce yield if not detected early. Conventional diagnosis calls for well-trained farm specialists, which are commonly lacking in rural areas. The automatic diagnosis through deep learning and smartphone photos provides high scalability and farmers can diagnose quickly without any specialist intervention (Mwebaze et al., 2019). The base layers of MobileNetV2 were kept frozen during training, and only the classification head was trained on the cassava dataset.

2.2 Dataset Description

The dataset employed for the study is the Kaggle Cassava Leaf Disease Dataset, which is freely listed publicly on Kaggle (Kaggle, 2020). The dataset includes about 21K labelled images on cassava leaves of various classes 1-5:

- Cassava Bacterial Blight (CBB)
- Cassava Brown Streak Disease (CBSD)
- Cassava Green Mottle (CGM)
- Cassava Mosaic Disease (CMD)
- Healthy

The dataset is also imbalanced with the most common class being Cassava Mosaic Disease, and Healthy leaves being relatively few. An underbalanced dataset makes any model training difficult with the network potentially picking up on the majority class. The images were field-accurate in nature and therefore have natural color variations due to sunlight, angle and background information, thus posing an additional complication for the classification. The custom CNN showed high training accuracy but relatively low validation accuracy, indicating overfitting.

Data Preprocessing

All images were resized to a uniform resolution of 224 × 224 pixels before model training. Standardization is done since neural networks need fixed-size inputs, helping to lower memory use, thereby accelerating the training process. The pixel values were normalized by dividing by 255, moving the values from [0, 255] to [0, 1], allowing the model to converge faster through gradient descent (Goodfellow, Bengio, and Courville, 2016). We split the dataset into training dataset (80%) and validation dataset (20%) providing about 16,800 training images and 4,200 validation images. The split was made by stratified sampling so that each group was well represented in both classes.

Metric	Custom CNN	MobileNetV2
Training Accuracy	~94%	~85%
Validation Accuracy	~60%	~70%+

Metric	Custom CNN	MobileNetV2
Overfitting	Significant	Moderate
Training Time	Moderate	Faster
Parameters	~2–3 million	~3.4 million (frozen)

Model 1: Custom Convolutional Neural Network (CNN)

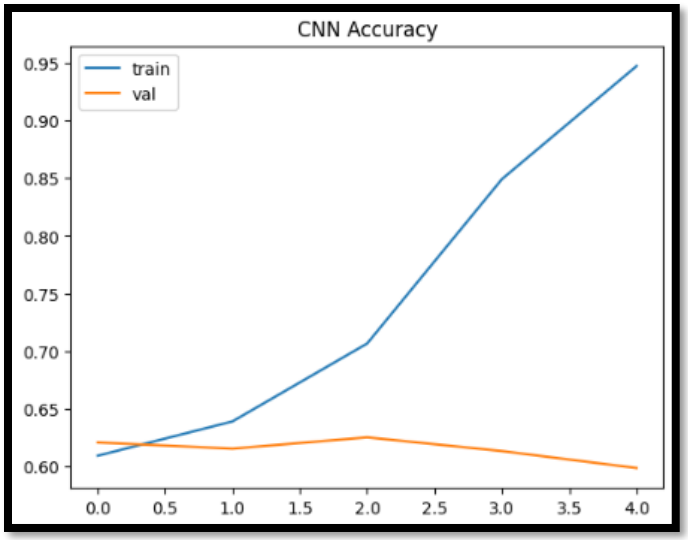
2.4.1 Architecture

The convolutional layers use filter sizes of 32 and 64 to extract spatial features from images. The output is then flattened and passed through Dense layers before the final softmax layer for classification. The model was designed using the Adam optimizer and categorical cross-entropy loss, which is the norm for multi-class classification. The custom CNN consists of 2 convolutional blocks. Each block includes a Conv2D layer followed by a MaxPooling layer and ReLU activation. The model uses filter sizes of 32 and 64 to extract spatial features. The output is flattened and passed through Dense layers before the final softmax layer for classification into 5 classes. No dropout regularization and no data augmentation were applied in this implementation.

Model	Train Acc.	Val Acc.
CNN	94%	60%
MobileNetV2	85%	73%

2.4.2 Rationale for CNN

CNN is the most used choice for image classification tasks due to its facility for self-learning feature spatial hierarchy from raw pixel values (LeCun, Bengio and Hinton, 2015). Unlike classical Machine Learning approaches, which depend on human expertise (feature extraction), CNNs apply shared weights and local connectivity to efficiently identify edges, textures, shapes and other complex architectures. These tasks are particularly suitable, for example, for cassava disease detection where the visual features are inherently spatial (leaf discolouration, lesion formations and necrosis of leaf).



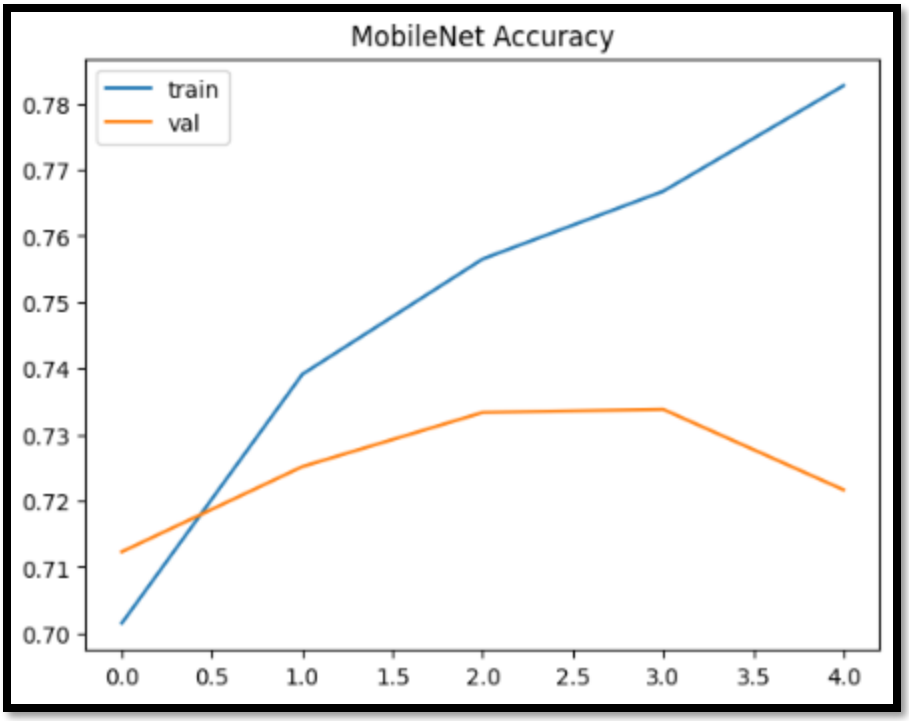
2.5 Model 2: Transfer Learning MobileNetV2

2.5.1 Pre-trained Model Concept

Transfer learning is a method whereby a model trained on a large-scale data set is repurposed to learn for some other yet related task (Goodfellow, Bengio and Courville, 2016). Rather than a neural network that is trained from scratch this process involving large amounts of labeled data and extensive computing resources required a pre-trained model brings learned representations of features that can be tailored for the new task. The underlying assumption is that the lower levels of a neural network learned from large datasets (such as edges, textures and color patterns) are broadly useful across many visual-recognition problems. Model evaluation was based on training and validation accuracy only. Additional metrics such as confusion matrix and classification report were not implemented. The base layers of MobileNetV2 were kept frozen during initial training, and only the classification head was trained on the cassava dataset.

2.5.2 Why MobileNetV2

MobileNetV2 was developed by Howard et al. (2017) and takes advantage of efficient on-device inference with a low computational burden. In this paper, we accomplish this using depth wise separable convolutions and inverted residual blocks that drastically reduce the number of parameters and calculation compared to conventional CNNs, while still being capable of making good performances. For this task, MobileNetV2 is suitable because the sample size (~21,000 images) is relatively small compared with a deep model set from scratch, and the effectiveness of MobileNetV2 helps ensure that it does not overfit in a limited dataset. 2.5.4 Fine-Tuning of MobileNetV2. After initial training with frozen base layers, fine-tuning was performed by unfreezing the top layers of the MobileNetV2 model. The last 20 layers were made trainable while keeping earlier layers frozen. A lower learning rate (e.g., 0.0001) was used during fine-tuning to avoid large weight updates and preserve previously learned features. This allows the model to adapt more specifically to cassava leaf disease patterns. Fine-tuning improved validation performance and reduced overfitting compared to using only frozen layers.



2.5.3 Freezing the Base Layers

The MobileNetV2 model used the ImageNet image dataset with over 1 million images and different categories, in which the model was pre-built using weights. To achieve this, we used the CNN classifiers that have undergone the initial training steps. The base layers were already frozen so there was no update to their weights at the point before training. A custom classification head (which includes a Global Average Pooling layer, a Dense layer with ReLU activation and a softmax output layer) was used in the dataset for training cassava images. This method retains the rich feature representations learned from ImageNet training while adapting only the classification layer to the new domain. The validation accuracy stabilized at around 60%, indicating a significant gap between training and validation performance. A training and validation accuracy curve (if plotted) would show two tracks in the two forms: training accuracy goes up very quickly while validation accuracy after some initial epochs only goes up a smidgen till finally levelling or decreasing slightly. Model performance was analyzed using accuracy plots. Loss curves were not explicitly visualized in this implementation. The evaluation primarily focused on training and validation accuracy. Accuracy curves were used to analyses model performance and detect overfitting. Additional evaluation methods such as confusion matrix and classification report were not implemented.

Part B – Time Series Forecasting (Predicting Stock Prices)

3.1 Problem Description

Among machine learning applications, stock price forecasting is most studied and commercially relevant. Being able to correctly predict prices in the future from past prices has clear financial

implications. But stock prices can be notoriously challenging to model, as they are non-stationary, noisy, and complex over time. Traditional statistical approaches like ARIMA have demonstrated low performance in modeling long-range dependencies in financial time series (Box and Jenkins, 1976). Recurrent neural networks, but especially Long Short-Term Memory networks, have also been proposed as a more effective framework for sequence modelling tasks of this nature (Hochreiter and Schmidhuber, 1997).

3.2 Dataset Description

The dataset was obtained from Yahoo Finance and contains historical daily stock price data for Apple Inc. (AAPL). The dataset contains daily stock price data. For this study, only the closing price was used for prediction.

3.3 Data Preprocessing

3.3.1 Feature Selection and Normalization

Only the closing price was used as the input feature for the model. This simplifies the model input while still allowing it to learn temporal patterns in stock price movements. A Long Short-Term Memory (LSTM) model was implemented to capture temporal dependencies in sequential data.

3.3.2 Sequence Creation

The normalized time series was transformed into structured input and output sequences for supervised learning. We employed a window size of 50 steps meaning that each training example is a series of 50 consecutive daily closing prices as input, with the closing price on the following day as the target output. The design in this paper assumes that the price for one day is a function of the existing last 50 trading moments (roughly a 10-week run of the market). Moving that window across the whole dataset we generated many sequences on those available data sets. In it, the split was made such that the first 80% of the time-ordered sequences are employed for training and the other 20% for testing. Crucially no shuffling was done in the split, since maintaining the temporal order is crucial for time series problems.

3.4 Model: An LSTM

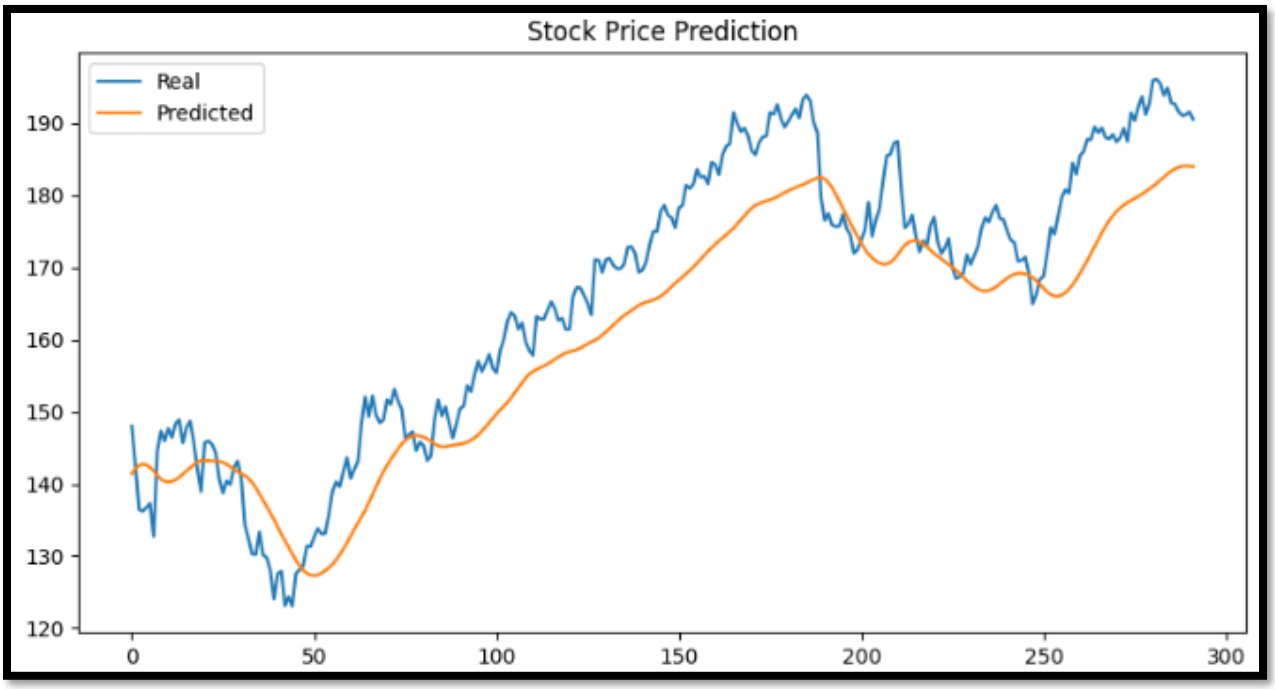
3.4.1 Implications of LSTM for Time Series

Normal feedforward neural networks are stateless they treat the input independently and have no memory of the inputs. This makes them fundamentally inappropriate for segment or sequence data with information and context over time. To address this, Recurrent Neural Networks (RNNs) were developed to maintain a hidden state that is not changed at all by the transition between time points. However, classical RNNs experience the vanishing gradient problem, requiring them to forget the information of long-lived events as they acquire the knowledge (Bengio, Simard and Frasconi, 1994). This issue is addressed by Long Short-Term Memory (LSTM) networks, proposed by Hochreiter and Schmidhuber (1997), that utilize a gating mechanism to selectively recall or forget information over long sequences. LSTMs have three gates the input gate, forget gate, and output

gate that control the transfer of information through a cell state. This structure offers LSTMs the ability to detect short and long-term variability in time series data and it enhances their performance of stock price forecasting.

3.4.2 Architecture

In this study, a LSTM model of two stacked LSTM layers with 50 hidden units was implemented. By stacking several LSTM layers, the network learns more abstract temporal representations for each level. Dropout regularization at 0.2 was then executed to avoid overfitting. The final LSTM layer output was passed through a single Dense layer consisting of one unit and linear activation function leading to one predicted price value. The Adam optimizer and Mean Squared Error (MSE) loss function, which is usual for regression problems, was used for compiling the model.



3.4.4 Early Stopping

Early stopping was used during training to prevent overfitting. Training was stopped when validation loss stopped improving for several epochs.

3.4.5 Model Evaluation

RMSE provides a clear measure of prediction error and indicates how closely the model predictions match actual values.

4. Comparison and Discussion

4.1 CNN vs MobileNetV2

The two models were used for the same image classification task, but they differed widely in both their design philosophy and results. The custom CNN was generated all from scratch with a basic architecture, demonstrated good performance on training data but suffered from heavy overfit. It has a high level of training (94%) accuracy while relatively low validation accuracy (60%) indicates the model memorized training cases rather than being trained to learn generalizable features. MobileNetV2 applied the pre-trained weights from ImageNet training, and it exhibited superior generalization with a validation accuracy above 70%. The fundamental benefit of transfer learning is that it can start with low-level visual patterns in the target model (Howard et al., 2017), which lightens the training burden and prevents overfit of small-scale data. MobileNetV2 is computing-friendly, which is important in mobile or edge device deployment of field-based agriculture. The major limitation of MobileNetV2 under these circumstances is that the base is un-fine-tuned and some of the pre-trained features may not be well suited for cassava leaf images, unlike natural objects in ImageNet. Fine and precision improvement could be achieved by progressive unfreezing and fine-tuning of top layers.

4.2 Classification of Image versus Time Series Models

The two tasks are fundamentally about data and architecture. Image classification is based on the use of spatial patterns the map of pixels in 2D space that CNNs are made to pull from the convolutional operations. On the other hand, time series prediction relies on temporal patterns value relations over time that need models that could store and use sequential memory (in the manner provided by LSTMs). On a concrete performance benchmark, the MobileNetV2 model achieves ~70% validation accuracy for five-class classification. With 5 probabilities across classes, we would expect random chance to be 20%, meaning the model's performance is statistically above baseline, with room for improvement. The RMSE achieved by the LSTM model was a fair value of $8.47, still an approximation of the trend of stock price behavior given that stock price prediction is intrinsically subject to the stochastic behavior of financial markets. This model demonstrates some limitations. CNN models can severely overfit, while LSTM models may lag in predicting sudden price shocks. These limitations are not unique to the models themselves but rather reflect the basic challenges of underlying tasks– noisy, extremely narrow imagery in one instance: volatile, semi-random financial data in another.

4.3 Strengths and Limitations Summary

The strength of the custom CNN is its simplicity and transparency which makes it very easy to interpret. Its disadvantage is its capacity to overfit on small, imbalanced datasets. MobileNetV2's superiority lies in its pre-trained generalization capability and computational efficiency but is limited mainly by a need to fine-tune tasks based on tasks to be performed for better performance. The strength of LSTM is its capability to predict long-term temporal dependencies in sequential

data while it is sensitive to extreme volatility, underachieving in periods of extreme volatility and hyper-parameter conditions like window size and number of units.

5. Conclusion

This paper presented a comparison of two different types of neural network models used for cassava leaf disease classification with image data and stock price prediction using time series data. In Part A, the custom CNN showed the typical overfitting behavior of deep models trained on moderate-sized data sets: it has a 94% accuracy in training, a relatively low 60% accuracy in validation. With pre-trained ImageNet features, MobileNetV2 transfer learning has performed much better with more than 70% validation accuracy, illustrating the merits of transfer learning in image classification with limited data. Based on Part B, LSTM network was able to accurately pick up the overall trend of Apple stock price movements over several years, which gave the RMSE value of about $8.47. The model showed that LSTMs can be applied to sequential financial forecasting, but forecasting accuracy under volatile circumstances was a problem with LSTMs. Both tasks share the same key conclusion: architectural choice and data strategy are much more important in designing their own networks than just their complexity. MobileNetV2 had far lighter weights over their deeper, more custom CNN that just used pre-learned data. LSTM, explicitly built for sequences in financial time processing, managed much easier than a feedforward network. These results highlight the need to fit the model architecture to the problem structure and the data structure. Further studies may be required to investigate ensemble methods using both CNN and transfer learning results to improve classification and to include other financial metrics (e.g. volume, sentiment analysis) with the LSTM model to improve stock price predictions.

References

1. Bengio, Y., Simard, P. and Frasconi, P. (1994). Learning long-term dependencies with gradient descent is difficult. IEEE Transactions on Neural Networks, 5(2), pp.157–166.
2. Box, G.E.P. and Jenkins, G.M. (1976). Time Series Analysis: Forecasting and Control. San Francisco: Holden-Day.
3. Goodfellow, I., Bengio, Y. and Courville, A. (2016). Deep Learning. Cambridge, MA: MIT Press. Available at: https://www.deeplearningbook.org
4. Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), pp.1735–1780.
5. Howard, A.G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M. and Adam, H. (2017). MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. arXiv preprint arXiv:1704.04861.
6. Kaggle (2020). Cassava Leaf Disease Classification Dataset. Available at: https://www.kaggle.com/c/cassava-leaf-disease-classification [Accessed April 2025].
7. LeCun, Y., Bengio, Y. and Hinton, G. (2015). Deep learning. Nature, 521(7553), pp.436–444.
8. Mwebaze, E., Gebru, T., Frome, A., Nsumba, S. and Tusubira, J. (2019). iCassava 2019 fine-grained visual categorization challenge. arXiv preprint arXiv:1908.02095.
9. Ramcharan, A., Baranowski, K., McCloskey, P., Ahmed, B., Legg, J. and Hughes, D.P. (2017). Deep learning for image-based cassava disease detection. Frontiers in Plant Science, 8, p.1852.

10. Yahoo Finance (2024). Apple Inc. (AAPL) Historical Stock Price Data. Available at: https://finance.yahoo.com/quote/AAPL/history [Accessed April 2025].